

BACKROOMS

A Living Memory Architecture for Continuous Knowledge Compression

[Redacted] | [Redacted] | 2026-06-28 | SPARKY / LOCAL CORTEX

The Claim

Most AI memory systems store or compress. BACKROOMS does both simultaneously — every abstraction it produces is a key back to the raw data that generated it.

Current approaches treat memory as retrieval: store a vector, fetch it when relevant. The original meaning and the retrieved meaning are assumed to be the same thing. They rarely are.

BACKROOMS takes a different position: meaning is not in the data — it is in the relationships between compressed representations of the data, built continuously over time, with full provenance intact.

The Problem with Current AI Memory

Vector databases retrieve but do not synthesize. They find what is similar — they do not discover what is significant. Significance requires repeated exposure, cross-referencing, and the emergence of structure that no single document contains.

RAG systems bolt memory onto inference as an afterthought. The model reads retrieved chunks and responds — but nothing about the retrieval process learns from what it finds. The memory does not grow. It does not connect. It does not abstract.

Most fundamentally: current AI memory has no concept of importance. Every stored item is equally retrievable. There is no mechanism by which frequently-relevant concepts rise, connect to others, and become something more than the sum of their source documents.

BACKROOMS is built on the premise that importance should be structural — discovered by the graph, not assigned by a human.

What BACKROOMS Is

BACKROOMS is a continuously running knowledge graph that processes incoming information and organizes it into four layers of increasing abstraction. It is always on. You do not look at it directly. It accumulates.

Nothing is lost at any layer. Every abstract synthesis at the top of the graph carries edges that trace back through intermediate concepts all the way to the exact source text chunk it originated from.

The abstraction and the original coexist — the higher node is a key to everything below it.

The Four Poles

BACKROOMS organizes all knowledge into four poles, each a higher compression of the one below:

Pole	What it contains
MEMORY POLE	Raw text chunks. The world coming in. Unprocessed source material.
CONCEPT POLE	AE nodes — atomic ideas extracted from chunks. Smallest meaningful unit.
AePI POLE	Emergent entities promoted from AEs. Not assigned — discovered when an AE accumulates 3+ inbound edges from other concepts.
ABPL POLE	Abstract syntheses. Spawned above clusters of AePIs when convergence is detected (4+ inbound structural edges). Re-derived every cycle to reflect current graph state.

Node Types and Promotion

The graph contains three node types, each promoted from the one below based purely on structural pattern — no model decides what is important:

AE (Atomic Element)

An idea extracted from a chunk by LOOKER. The smallest meaningful unit in the graph. Meaning lives not in the node itself but in its edges — what it connects to determines what it becomes.

AePI (Emergent Entity)

An AE that has accumulated 3 or more inbound dependency edges from other AEs. It is not converted — it is promoted in place. The AE itself becomes the AePI. Its contributing children are sealed inside it, permanently. AePI promotions never undo. Once an idea has proven its significance through structural connectivity, that significance is permanent.

ABPL (Abstract Layer)

A new node spawned above a cluster of AePIs. Not a conversion of any existing node — a fresh entity created when PROMOTER2 detects an AePI with 4 or more inbound structural edges from other AePIs (an attractor). The ABPL draws cluster member edges from the attractor and all its sources, then SYNTHESIZER writes a 2-3 sentence plain-language synthesis of the shared insight. ABPLs are wiped and re-derived every cycle so they always reflect the current state of the AePI graph beneath them.

The Agents

Six agents run in sequence each cycle. No agent does more than one job:

Agent	Role
LOOKER	Reads 6 chunks at a time from the memory pole. Extracts ideas, calls LABELER internally. Runs as background thread.
LABELER	Stamps each extracted idea as an AE node. Called by LOOKER.
LINKER1	AE pass. Compares unlinked AE pairs. Creates direct dependency edges only — shared theme alone does not qualify.
PROMOTER1	Pure pattern detection. AEs with 3+ inbound edges are promoted to AePI. No model involved.
LINKER2	AePI pass. Stricter than LINKER1 — structural dependency only. Tags edges separately so PROMOTER2 cannot confuse them with LINKER1 edges.
PROMOTER2	Wipes all ABPLs, finds AePI attractors (4+ inbound LINKER2 edges), spawns fresh ABPLs above each attractor cluster.
SYNTHESIZER	Finds blank ABPLs, reads their cluster members, writes a 2-3 sentence plain-language synthesis of the shared insight.

One Cycle

Each cycle runs the full pipeline and then repeats indefinitely:

```
LOOKER – reads 6 chunks → extracts AE nodes LINKER1 – compares new AE pairs → dependency edges PROMOTER1 – detects 3+ inbound edges on AEs → promotes to AePl LINKER2 – compares AePl pairs → structural edges PROMOTER2 – wipes ABPLs, detects attractors → spawns new ABPLs SYNTHESIZER– writes plain synthesis for each blank ABPL → repeat
```

The system does not stop. There is no query that triggers it. Everything that enters the memory pole is eventually processed, promoted if significant, and abstracted if it converges with other significant concepts.

The Key Property: Lossless Abstraction

Every node at every layer maintains full edge provenance back to its source. An ABPL synthesis at the top of the graph is not a replacement for the ideas beneath it — it is an index into them. Read the synthesis to understand the cluster. Follow the edges to read every AePI it synthesizes. Follow

those edges to read every AE that contributed. Follow those edges to read the original text chunks.

Nothing is discarded. Compression and provenance coexist at every level. The graph grows richer with every cycle, not more lossy.

Most systems compress and discard. BACKROOMS compresses and preserves — the abstraction and its origin coexist permanently in the same graph.

The four-pole structure is not fixed. Additional poles can be introduced above the ABPL layer to achieve further levels of compression and abstraction — each new pole operating on the synthesized output of the one below it. The architecture scales vertically: more poles, higher abstraction, same provenance guarantee all the way to the source.

Current State

BACKROOMS is running live as of 2026-06-26, processing a corpus of personal journal entries and philosophical notes (213 chunks). Current graph state:

Metric	Value
AE nodes	~119
AePI nodes	~32
ABPL nodes	Growing — convergence-based promotion active
Edges	~705
Model	Qwen 2.5 1.5B Instruct (shared across all agents)
Graph store	SQLite WAL mode
Hardware	RTX 3050 Ti (4GB VRAM) — consumer hardware

Note on the AE:AePI ratio: The current corpus is Jake's personal journal — philosophically coherent, high cross-reference density. The high promotion rate is expected and will normalize with more diverse input. Threshold tuning is deferred until varied data is processed.

IP Notice

BACKROOMS — the living memory architecture described in this document, including the four-pole graph structure, promotion mechanics (AE → AePI → ABPL), lossless abstraction with full edge provenance, and the six-agent pipeline — is original work by [Redacted] ([Redacted]), developed under the SPARKY / LOCAL CORTEX project beginning 2026-06-21. Contact the author before using, adapting, or building on this architecture.

First documented: 2026-06-28